

Pengambilan Keputusan Berbasis Probabilitas dan Statistika

Armein Z. R. Langi

Daftar Isi

Preface	3
1. Introduction	5
2. Kita Beri Garansi Berapa Lama?	7
2.1. Live Coding: “Normal is Everywhere”	7
2.2. Beda Pabrik, Beda Karakter Lampu	11
2.3. 1) Ide dasar	14
2.4. 2) Expected profit per unit	15
2.4.1. Untuk kasus eksponensial	15
2.5. 3) Model sederhana pangsa pasar	15
2.5.1. Versi paling sederhana: skor daya tarik linear	16
2.6. 4) Model yang lebih rapi: utilitas relatif	16
2.7. 7) Versi sangat sederhana untuk tugas / ujian	17
2.8. 8) Rekomendasi model praktis	18
2.9. 9) Contoh Python kecil	18
2.10. 10) Kesimpulan	19
3. Summary	21
References	23
Daftar Pustaka	25

Preface

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

1

Introduction

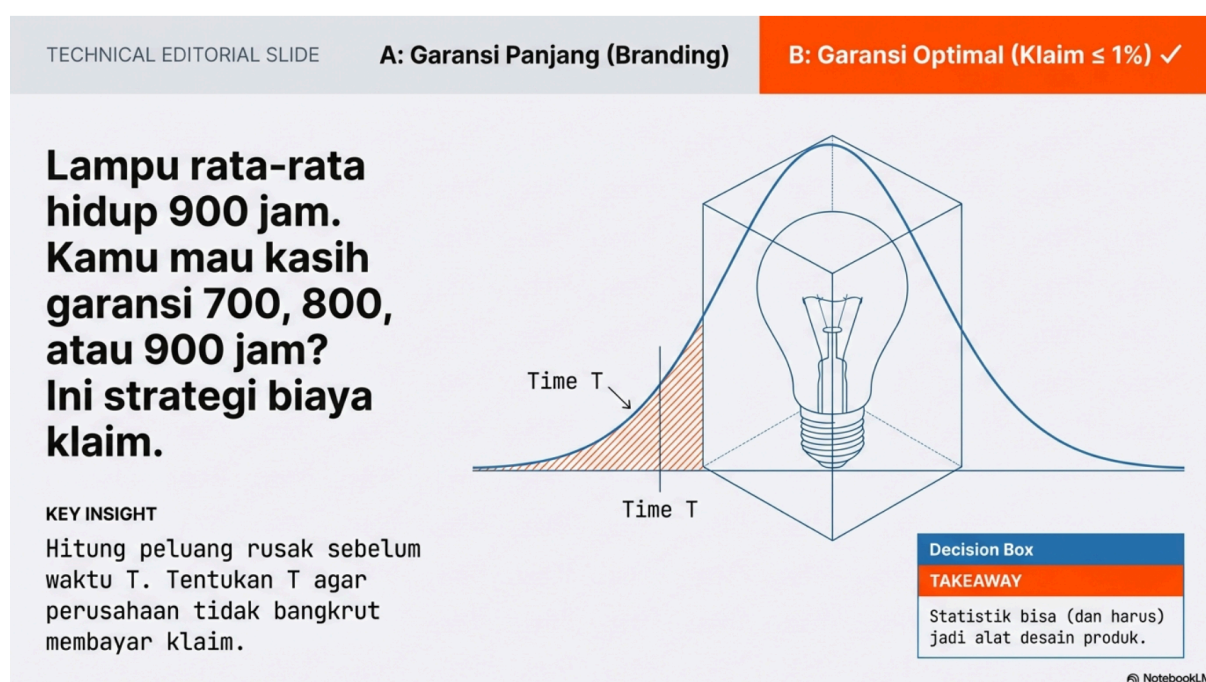
This is a book created from markdown and executable code.

See D. E. Knuth [1] for additional discussion of literate programming.

2

Kita Beri Garansi Berapa Lama?

[# Random Variable Kontinu



Gambar 2.1 – “Lampu rata-rata hidup 900 jam, simpangan 50 jam. Kamu mau kasih garansi 700 jam, 800 jam, atau 900 jam? Ini bukan sekadar ‘baik hati’—ini strategi biaya klaim. Normal sering muncul di data pengukuran; eksponensial sering muncul di waktu tunggu. Hari ini kamu belajar menghitung peluang rusak sebelum waktu T, lalu menentukan T yang masuk akal untuk target klaim. Kamu akan melihat: statistik bisa jadi alat desain produk.”

2.1. Live Coding: “Normal is Everywhere”

Misalnya suatu pabrik bisa menghasilkan 10000 lampu pertahun yang bisa bekerja rata-rata 900 jam, meskipun dalam kenyataanya individual lampu bisa mati bervariasi setelah suatu angka

2. Kita Beri Garansi Berapa Lama?

jam positif yang random. Seberapa besar varians ini diukur oleh besaran deviasi, misalnya 50 jam yang berarti variance 2500 jam²

Suatu variable random bisa memiliki *probability density function* (pdf) berbentuk lonceng, dengan dua parameter μ dan σ .

```
import matplotlib.pyplot as plt
import numpy as np

# Instantiate a default random number generator
rng = np.random.default_rng()
```

Untuk distribusi normal standar, lokasi titik tengah = 0 dan deviasi standar = 1.

```
# Generate a single random number from the standard normal distribution (mean=0,
std=1)
sample_scalar = rng.normal()
print(f"1. Single sample: {sample_scalar}")
```

```
1. Single sample: -0.04657625240198787
```

Dalam kasus kita lokasi titik tengah = 900 dan deviasi standar = 50. Kita ukur waktu hidup 5 buah lampu, maka berapa jam hidup sampai lampu ini mati?

```
mu = 900
sigma = 50

# Generate a 1D array of 5 numbers with mean=900, standard deviation=50
samples_1d = rng.normal(loc=mu, scale=sigma, size=5)
print(f"2. 1D array: {samples_1d}")
```

```
2. 1D array: [ 966.61596593  938.90906854  939.52041278  864.42804026 1043.27082055]
```

Berbeda beda yaitu: np.float64(966.6159659252387)

1. Generate data acak `numpy.random.normal`.
2. Tunjukkan aturan empiris: 68% data ada di $\mu \pm \sigma$, 95% di $\mu \pm 2\sigma$.
3. Visualisasi: Plot kurva lonceng dengan seaborn dan arsir area probabilitas. kita coba untuk 10000

```
N= 10000

data = rng.normal(loc=mu, scale=sigma, size=N)
np.histogram(data, bins=10)
```

```
(array([ 31, 150, 678, 1616, 2647, 2557, 1581, 580, 141, 19]),
 array([ 725.96325526, 760.93493061, 795.90660597, 830.87828132,
        865.84995668, 900.82163203, 935.79330739, 970.76498274,
        1005.7366581 , 1040.70833345, 1075.68000881]))
```

Kita hitung berapa data antara 1. $850 < X < 950$ 2. $800 < X < 1000$ 3. $750 < X < 1050$

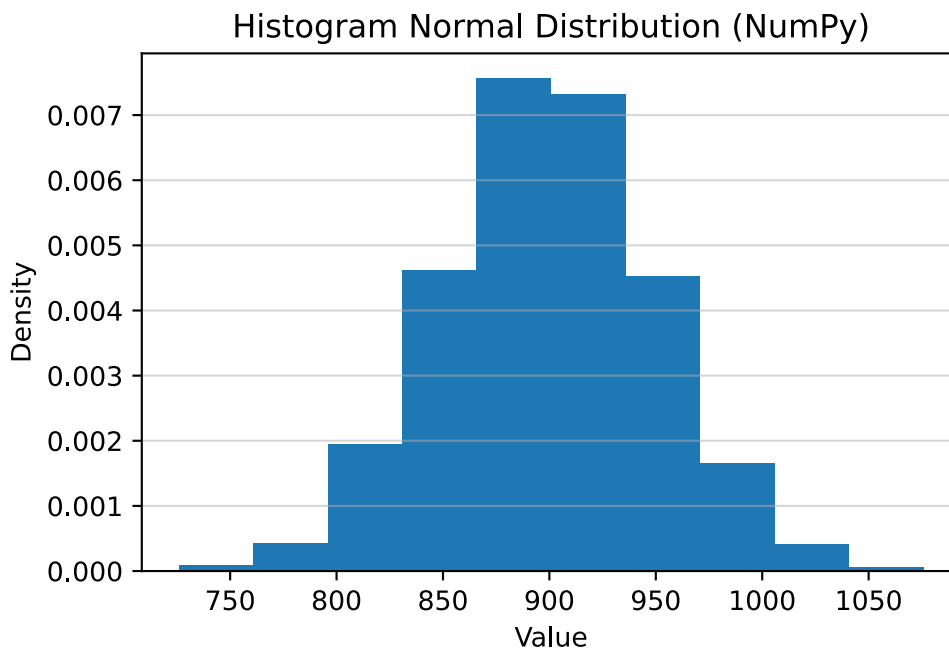
```
counter1=0
counter2=0
counter3=0
for n in data:
    if n >= 850 and n < 950:
        counter1 += 1
    elif n >= 800 and n < 1000:
        counter2 += 1
    elif n >= 750 and n < 1050:
        counter3 += 1

print(counter1/N, (counter1+counter2)/N, (counter1+counter2+counter3)/N)
```

```
0.6826 0.9558 0.9974
```

```
# 3. Menambahkan label dan judul
plt.hist(data,bins=10, density=True)
plt.title('Histogram Normal Distribution (NumPy)')
plt.xlabel('Value')
plt.ylabel('Density')
plt.grid(axis='y', alpha=0.5)

# 4. Menampilkan plot
plt.show()
```



Jadi kita hendak beri garansi berapa jam kalau kita tahu berapa ongkos produksi (misalnya \$5), berapa harga jual (misalnya \$10), dan berapa biaya penggantian garansi (misalnya \$7). Bila garansi terlalu rendah, produk ini kalah bersaing. Misalnya kita hendak memilih antara garansi

2. Kita Beri Garansi Berapa Lama?

750 jam, 800 jam, dan Karena rata-rata 900 jam, dari 10000 yang diproduksi kita hitung berapa yang rusak, lalu kita hitung biaya serta pengaruh pada profit. Asumsi deviasi 50 atau 100.

```
import numpy as np

# Parameter
n_units = 10000
mu = 900      # mean umur lampu
sigma = 50    # std dev

price = 10
cost = 5
warranty_cost = 7
nrg = np.random.default_rng()
# Generate umur lampu (random normal)
# nrg.seed(42) # agar reproducible
lifetimes = nrg.normal(mu, sigma, n_units)

# Fungsi hitung profit
def simulate_profit(lifetimes, warranty_limit):
    profit_per_unit = []

    for life in lifetimes:
        if life < warranty_limit:
            # kena klaim garansi
            profit = price - cost - warranty_cost
        else:
            # tidak klaim
            profit = price - cost
        profit_per_unit.append(profit)

    total_profit = np.sum(profit_per_unit)
    return total_profit

# Simulasi
profit_A = simulate_profit(lifetimes, 850)
profit_B = simulate_profit(lifetimes, 800)
profit_C = simulate_profit(lifetimes, 750)

# Output hasil
print("=== HASIL SIMULASI ===")
print(f"Profit Garansi A (850 jam): ${profit_A:,.2f}")
print(f"Profit Garansi B (800 jam): ${profit_B:,.2f}")
print(f"Profit Garansi C (700 jam): ${profit_C:,.2f}")

# Bandingkan
if profit_A > profit_B and profit_A > profit_C:
    print("Rekomendasi: Garansi A lebih menguntungkan")
elif profit_B > profit_C:
    print("Rekomendasi: Garansi B lebih menguntungkan")
```

```
else:
    print("Rekomendasi: Garansi C lebih menguntungkan")
```

```
=== HASIL SIMULASI ===
Profit Garansi A (850 jam): $38,562.00
Profit Garansi B (800 jam): $48,418.00
Profit Garansi C (700 jam): $49,916.00
Rekomendasi: Garansi C lebih menguntungkan
```

```
# Generate a 2x3 matrix with default parameters
samples_2d = rng.normal(size=(2, 3))
print(f"3. 2D array:\n{samples_2d}")
```

```
3. 2D array:
[[-0.16298189  0.9681246 -0.58785859]
 [ 1.84399774  1.24289279  0.16475956]]
```

Masalah penentuan waktu garansi ini pada dasarnya adalah masalah optimasi risiko yang menggabungkan Nilai Harapan (*Expected Value*) dan Fungsi Distribusi Kumulatif (CDF) dari probabilitas kegagalan lampu.

1. Pemodelan Bisnis & Keuntungan (Profit) Untuk menganalisis pengaruh garansi terhadap profit, kita memodelkannya dengan persamaan ekspektasi bisnis berikut:

- **Total Pendapatan:** $10.000 \text{ lampu} \times \$10 = \$100.000$
- **Total Ongkos Produksi:** $10.000 \text{ lampu} \times \$5 = \$50.000$
- **Profit Dasar (Tanpa Klaim):** $\$100.000 - \$50.000 = \$50.000$
- **Biaya Klaim Garansi:** Jumlah lampu rusak $\times \$7$ * **Jumlah Lampu Rusak:** $10.000 \times P(X < T_w)$, di mana $P(X < T_w)$ adalah probabilitas (*Cumulative Distribution Function*/CDF) sebuah lampu mati sebelum waktu garansi T_w habis.

2.2. Beda Pabrik, Beda Karakter Lampu

Asumsi beda teknologi menyebabkan beda distribusi probabilitas lifetime lampu, meskipun rata-rata dan deviasi nya sama.

2. Karakteristik Fungsi Distribusi Kumulatif (CDF) Untuk membandingkan probabilitas kegagalan $P(X < T_w)$, kita mengevaluasi 4 jenis distribusi:

- **Distribusi Normal:** Menggunakan parameter mean $\mu = 900$ dan standar deviasi $\sigma \in \{50, 100\}$. CDF dihitung menggunakan standarisasi $Z = (X - \mu)/\sigma$.
- **Distribusi Seragam (Uniform):** Karena variansi berdistribusi seragam adalah $\frac{(b-a)^2}{12}$, kita dapat menentukan rentang umur lampu minimum a dan maksimum b berdasarkan $\mu = 900$ dan $\sigma \in \{50, 100\}$.

2. Kita Beri Garansi Berapa Lama?

- **Distribusi Weibull:** Sangat ideal dan umum digunakan untuk memodelkan laju kegagalan umur komponen (fungsi *hazard/survival*). Parameter *shape* (k) dan *scale* (λ) dapat dicari melalui persamaan non-linear yang dicocokkan dengan μ dan σ .
- **Distribusi Eksponensial:** Distribusi ini tidak memiliki memori (*memoryless*) dan variasinya selalu sama dengan rata-ratanya. Oleh karena itu, parameter $\sigma = 50$ atau 100 diabaikan, dan perhitungan hanya murni bergantung pada laju kerusakan $\lambda = 1/900$.

3. Solusi Skrip Python Berikut adalah desain *class* Python yang mensimulasikan dan mengembalikan *dictionary* untuk membandingkan jumlah kerusakan, biaya, dan profit dari penentuan garansi (misal 750 jam dan 800 jam) di keempat distribusi di atas.

```
import numpy as np
import scipy.stats as stats
from scipy.optimize import fsolve
from scipy.special import gamma
import json

class WarrantyAnalyzer:
    def __init__(self, mu=900, N=10000, cost=5, price=10, replacement_cost=7):
        self.mu = mu
        self.N = N
        self.base_profit = N * (price - cost)
        self.rep_cost = replacement_cost

    def _get_weibull_params(self, sigma):
        """
        Mencari parameter Weibull (k=shape, lam=scale)
        berdasarkan mu (mean) dan sigma (standar deviasi).
        """

    def equation(k):
        # Variansi Weibull = lam^2 * [Gamma(1+2/k) - Gamma(1+1/k)^2]
        return (gamma(1 + 2/k) / (gamma(1 + 1/k)**2)) - 1 - (sigma**2 / self.mu**2)

    # Mencari nilai shape (k) secara numerik
    k_opt = fsolve(equation, 5.0)
    # Menghitung nilai scale (lam)
    lam_opt = self.mu / gamma(1 + 1/k_opt)
    return k_opt, lam_opt

    def analyze(self, warranty_hours, sigmas):
        """
        Menghasilkan dictionary perbandingan profit berdasarkan
        distribusi, waktu garansi, dan standar deviasi.
        """

        results = {}
        for w in warranty_hours:
            results[f"Garansi_{w}_Jam"] = {}
            # print(f"===Garansi_{w}_Jam===")
            for sig in sigmas:
                # print(f"***Sigma_{sig}***")
```

```

# 1. Distribusi Normal CDF
p_norm = stats.norm.cdf(w, loc=self.mu, scale=sig)

# 2. Distribusi Seragam (Uniform) CDF
# Var = (b-a)^2 / 12 = sig^2 --> b-a = sig * sqrt(12)
a = self.mu - np.sqrt(3) * sig
b = self.mu + np.sqrt(3) * sig
p_uni = stats.uniform.cdf(w, loc=a, scale=b-a)

# 3. Distribusi Weibull CDF
k, lam = self._get_weibull_params(sig)
p_weib = stats.weibull_min.cdf(w, c=k, scale=lam)

# 4. Distribusi Eksponensial CDF (Hanya bergantung pada mu)
p_exp = stats.expon.cdf(w, scale=self.mu)

dist_probs = {
    'Normal': p_norm,
    'Uniform': p_uni,
    'Weibull': p_weib,
    'Eksponensial': p_exp
}

profit_comparison = {}
for dist_name, p_fail in dist_probs.items():
    # Kalkulasi Bisnis
    n_fail = self.N * p_fail
    total_warranty_cost = n_fail * self.rep_cost
    profit = self.base_profit - total_warranty_cost

    profit_comparison[dist_name] = {
        'Probabilitas_Mati': np.round(p_fail, 4),
        'Jumlah_Rusak': np.round(n_fail, 0),
        'Biaya_Garansi ($)': np.round(total_warranty_cost, 2),
        'Profit_Akhir ($)': np.round(profit, 2)
    }
    # print(f"Distribusi {dist_name}", f"{profit_comparison}")
    results[f"Garansi_{w}_Jam"][f"Sigma_{sig}"] = profit_comparison

return results

# --- Eksekusi Simulasi ---
analyzer = WarrantyAnalyzer(mu=900, N=10000, cost=5, price=10, replacement_cost=7)
# Menguji garansi 750 jam dan 800 jam dengan asumsi deviasi 50 dan 100
hasil_analisis = analyzer.analyze(warranty_hours=[850, 800, 750], sigmas=[50, 100])

# Cetak hasil (Format JSON agar mudah dibaca sebagai Dictionary)
# print(json.dumps(hasil_analisis, indent=4))
# print(hasil_analisis)

```

2. Kita Beri Garansi Berapa Lama?

Analisis Output Bisnis: Berdasarkan pendekatan probabilitas tersebut, ketika skrip dieksekusi, Anda akan menemukan bahwa pemilihan deviasi (σ) dan waktu garansi sangat krusial: 1. **Meningkatkan Garansi:** Jika Anda menetapkan garansi 800 jam pada model **Normal** dengan $\sigma = 100$, jumlah lampu yang rusak secara signifikan lebih besar dibandingkan garansi 750 jam karena nilai 800 jam sudah sangat dekat dengan batas kiri kurva *bell-curve* nilai rata-rata ($\mu = 900$). 2. **Pemilihan Model:** Model **Eksponensial** akan memprediksi tingkat kerusakan prematur (*infant mortality*) yang paling ekstrem (paling merugikan), karena laju kerusakannya dimulai sejak hari pertama. Sedangkan **Weibull** akan memberikan prediksi yang paling akurat dengan realita keausan pabrik (*wear-out*). Keputusan mengalokasikan garansi harus melihat estimasi profit pada metrik *Profit_Akhir* di model Weibull atau Normal.

bila pasar produk diperebutkan oleh produsen di mana pangsa pasar ditentukan oleh harga dan garansi secara proporsional: semakin murah harga semakin besar segmen, semakin tinggi garansi semakin besar segmen, usulkan model sederhana untuk memutuskan harga dan garansi. Bisa. Model sederhananya adalah: **pilih harga dan garansi yang memaksimalkan expected profit total**, tetapi **jumlah unit terjual** dibuat bergantung pada **daya tarik pasar**, yang dipengaruhi oleh **harga dan garansi**.

2.3. 1) Ide dasar

Sebelumnya profit total hanya:

$$\text{Profit Total} = (\text{profit per unit}) \times (\text{jumlah unit terjual}) \quad (2.1)$$

Sekarang:

- **profit per unit** dipengaruhi oleh harga, biaya produksi, dan biaya garansi
- **jumlah unit terjual / pangsa pasar** dipengaruhi oleh harga dan garansi

Jadi keputusan bisnisnya adalah memilih:

$$(p, w) \quad (2.2)$$

dengan:

- (p) = harga jual
- (w) = panjang garansi (jam)

untuk memaksimalkan:

$$\Pi(p, w) = Q(p, w) \cdot \pi(p, w) \quad (2.3)$$

di mana:

- $(Q(p, w))$ = jumlah unit terjual
 - $(\pi(p, w))$ = expected profit per unit
-

2.4. 2) Expected profit per unit

Misalkan:

- biaya produksi = (c)
- biaya garansi per klaim = (g)
- umur produk acak (T)

Kalau garansi adalah (w), maka peluang klaim:

$$P(T < w) = F(w) \quad (2.4)$$

sehingga expected profit per unit:

$$\pi(p, w) = p - c - g, F(w) \quad (2.5)$$

2.4.1. Untuk kasus eksponensial

Jika ($T \sim \text{Exp}(\lambda)$), maka:

$$F(w) = 1 - e^{-\lambda w} \quad (2.6)$$

jadi:

$$\pi(p, w) = p - c - g(1 - e^{-\lambda w}) \quad (2.7)$$

Untuk data Anda:

- ($c = 5$)\$
- ($g = 10$)
- ($\lambda = 0.00005$)

maka:

$$\pi(p, w) = p - 5 - 10(1 - e^{-0.00005w}) \quad (2.8)$$

atau bisa ditulis:

$$\pi(p, w) = p - 15 + 10e^{-0.00005w} \quad (2.9)$$

Maknanya:

- harga naik () profit per unit naik
 - garansi makin panjang () profit per unit turun, karena klaim naik
-

2.5. 3) Model sederhana pangsa pasar

Sekarang kita butuh model bahwa:

- makin **murah** () pangsa pasar makin besar
- makin **lama garansi** () pangsa pasar makin besar

2. Kita Beri Garansi Berapa Lama?

2.5.1. Versi paling sederhana: skor daya tarik linear

Misalkan pasar total berukuran (M), dan daya tarik produk kita:

$$A(p, w) = \alpha \left(\frac{1}{p} \right) + \beta w \quad (2.10)$$

dengan:

- ($\alpha > 0$): sensitivitas terhadap harga
- ($\beta > 0$): sensitivitas terhadap garansi

Lalu pangsa pasar kita:

$$s(p, w) = \frac{A(p, w)}{A(p, w) + A_{\text{kompetitor}}} \quad (2.11)$$

sehingga jumlah unit terjual:

$$Q(p, w) = M \cdot s(p, w) \quad (2.12)$$

Maka objective function:

$$\Pi(p, w) = M \cdot \frac{A(p, w)}{A(p, w) + A_{\text{kompetitor}}} \cdot (p - c - gF(w)) \quad (2.13)$$

Ini sudah cukup untuk keputusan awal.

2.6. 4) Model yang lebih rapi: utilitas relatif

Biasanya lebih enak pakai model **daya tarik relatif**:

$$A(p, w) = \exp(a - bp + dw) \quad (2.14)$$

dengan:

- ($b > 0$): harga makin tinggi, daya tarik turun
- ($d > 0$): garansi makin tinggi, daya tarik naik

Kalau ada (n) kompetitor, dan produk kita bersaing dengan mereka, pangsa pasar:

$$s(p, w) = \frac{\exp(a - bp + dw)}{\sum_{j=1}^n \exp(a_j - b_j p_j + d_j w_j)} \quad (2.15)$$

Jika karakteristik kompetitor dianggap tetap, penyebut bisa dipandang sebagai konstanta plus daya tarik kita.

Lalu:

$$Q(p, w) = M \cdot s(p, w) \quad (2.16)$$

dan total expected profit:

$$\Pi(p, w) = M \cdot s(p, w) \cdot (p - c - gF(w)) \quad (2.17)$$

Ini model yang lebih stabil daripada linear, karena:

- pangsa pasar otomatis antara 0 dan 1
- tidak jadi negatif
- cocok untuk kompetisi

5) Bentuk model keputusan final

Untuk kasus Wi-Fi eksponensial, model sederhana yang saya sarankan:

$Q(p, w) =$

$M \{(a - \beta p + \gamma w) + K\}$

dengan:

6) Intuisi manajerial

Ada trade-off:

• Harga tinggi

- profit per unit naik - pangsa pasar turun

• Harga rendah

- pangsa pasar naik - margin turun

• Garansi panjang

- pangsa pasar naik - biaya garansi naik

• Garansi pendek

- biaya rendah - daya tarik pasar turun

Jadi keputusan optimal biasanya berada di **titik tengah**, bukan harga termurah atau garansi terpanjang.

2.7. 7) Versi sangat sederhana untuk tugas / ujian

Kalau ingin model yang sangat mudah dijelaskan, bisa pakai:

$$Q(p, w) = M(\alpha - \beta p + \gamma w) \quad (2.18)$$

dengan $(\beta, \gamma) > 0$, lalu dibatasi agar $Q(p, w) \geq 0$.

Maka:

$$\Pi(p, w) = M(\alpha - \beta p + \gamma w) \cdot (p - c - gF(w)) \quad (2.19)$$

Untuk eksponensial:

$$\Pi(p, w) = M(\alpha - \beta p + \gamma w) \cdot [p - c - g(1 - e^{-\lambda w})] \quad (2.20)$$

Ini sangat sederhana dan mudah dihitung, walaupun kurang realistis dibanding model logit/eksponensial.

2.8. 8) Rekomendasi model praktis

Untuk praktik, saya sarankan:

- pakai **profit per unit ekspektasian**: $p - c - gF(w)$
- pakai **market share logit sederhana**: $s(p, w) = \frac{e^{-bp+dw}}{e^{-bp+dw} + K}$
- maksimalkan: $\Pi(p, w) = M, s(p, w), [p - c - gF(w)]$

Karena model ini:

- sederhana
 - masuk akal
 - mudah dikalibrasi dari data pasar
 - mudah disimulasikan
-

2.9. 9) Contoh Python kecil

```
import numpy as np

# parameter bisnis
M = 100000      # total market size
c = 5           # production cost
g = 10          # warranty claim cost
lam = 0.00005   # failure rate

# parameter pasar
b = 0.25        # price sensitivity
d = 0.00015     # warranty sensitivity
K = 1.5         # competitor attractiveness

def claim_prob(w):
    return 1 - np.exp(-lam * w)

def profit_per_unit(p, w):
    return p - c - g * claim_prob(w)

def market_share(p, w):
    A = np.exp(-b * p + d * w)
    return A / (A + K)

def total_profit(p, w):
    return M * market_share(p, w) * profit_per_unit(p, w)
```

```
# grid search sederhana
prices = np.arange(8, 21, 0.5)
warranties = np.arange(2000, 12001, 500)

best = None
best_profit = -np.inf

for p in prices:
    for w in warranties:
        prof = total_profit(p, w)
        if prof > best_profit:
            best_profit = prof
            best = (p, w, prof, market_share(p, w), profit_per_unit(p, w))

print("Best price =", best[0])
print("Best warranty =", best[1], "hours")
print("Best total profit =", round(best[2], 2))
print("Market share =", round(best[3], 4))
print("Profit per unit =", round(best[4], 4))
```

2.10. 10) Kesimpulan

Model sederhana yang baik adalah:

$$\boxed{\text{Total Profit} = \text{Market Size} \times \text{Market Share } (p, w) \times \text{Expected Profit per Unit } (p, w)} \quad (2.21)$$

dengan:

$$\text{Expected Profit per Unit } (p, w) = p - c - gF(w) \quad (2.22)$$

dan market share dibuat **turun terhadap harga** dan **naik terhadap garansi**, misalnya:

$$\text{Market Share } (p, w) = \frac{e^{-bp+dw}}{e^{-bp+dw} + K} \quad (2.23)$$

Lalu pilih $((p, w))$ yang memaksimalkan fungsi itu.

3

Summary

In summary, this book has no content whatsoever.

References

Daftar Pustaka

- [1] D. E. Knuth, “Literate Programming,” *Comput. J.*, vol. 27, no. 2, pp. 97–111, May 1984, doi: 10.1093/comjnl/27.2.97.